

Relazione Progetto Finale PISSIR:

Sistema "BitPub"

Il presente documento costituisce la relazione tecnica finale per il progetto del corso PISSIR. Il sistema, denominato **BitPub**, consiste in una piattaforma distribuita IoT e Cloud per la gestione, il monitoraggio in tempo reale e l'organizzazione di tornei per giochi da pub (biliardo, calciobalilla, freccette).

01. Specifiche Funzionali

In accordo con le direttive del progetto, le funzionalità richieste sono state suddivise in due domini di competenza principali, atti a garantire resilienza e operatività in contesti reali di pub e sale giochi.

1.1 Controllo Locale

- **Rilevamento Eventi in Tempo Reale:** Il sistema deve interfacciare i sensori installati sui tavoli da gioco (es. buche del biliardo, sensori di porta del calciobalilla, bersagli elettronici delle freccette) per rilevare i punteggi e gli stati del gioco istantaneamente.
- **Resilienza Offline:** In caso di caduta della connessione internet del locale, il nodo locale (Edge) deve continuare a processare gli eventi, bufferizzarli e garantire il normale svolgimento delle partite.
- **Motore di Regole Locale:** Validazione immediata delle mosse e calcolo dei punteggi parziali eseguito in locale per ridurre la latenza.

1.2 Accessibilità via Internet

- **Sincronizzazione Cloud:** I dati delle partite, una volta consolidati dal nodo Edge, devono essere sincronizzati con un'architettura Cloud centralizzata.
- **Gestione Utenti e Tornei:** Accesso remoto per i giocatori per visualizzare le proprie statistiche storiche, registrarsi a tornei e visualizzare le classifiche globali (Leaderboards).
- **Pannello di Controllo:** Una dashboard amministrativa via WebApp che consente ai gestori dei locali (Locales) di monitorare le proprie istanze di gioco (Game Instances) e configurare i tavoli.

02. Analisi Tecnologica

L'ambiente operativo del sistema presenta vincoli specifici che hanno guidato le scelte progettuali, in particolare per quanto riguarda la topologia di rete e il flusso dei dati.

2.1 Installazione in Reti Private

I pub e le sale giochi operano tipicamente dietro NAT (Network Address Translation) e firewall restrittivi. Non è ipotizzabile richiedere ai gestori di configurare port forwarding o regole firewall specifiche. Pertanto, il paradigma di comunicazione non può basarsi su un Cloud che "chiama" il locale (Push da Cloud a Edge via HTTP), ma deve prevedere che il nodo Edge all'interno della rete privata inizializzi la connessione verso l'esterno, mantenendo un canale di comunicazione bidirezionale persistente.

2.2 Accesso dalla Rete Pubblica

Per garantire un accesso unificato dalla rete pubblica (giocatori da smartphone, gestori da casa), è stata progettata un'architettura a Microservizi esposta tramite un API Gateway. Questo pattern nasconde la complessità della topologia interna, offrendo un singolo punto di accesso (Single Point of Entry) e delegando la sicurezza (autenticazione JWT) al Gateway stesso prima che il traffico raggiunga i servizi di dominio.

2.3 Traffico Dati e Real-Time

Il traffico generato dai sensori (es. BilliardsEventGenerator, FoosballEventGenerator) è caratterizzato da payload molto piccoli ma alta frequenza (burst di eventi durante una partita). I requisiti real-time sono stringenti a livello locale (feedback immediato al giocatore), ma meno critici a livello Cloud (aggiornamento delle statistiche a fine partita o in background). Questo giustifica una separazione architetturale tra il bus di eventi locale e la sincronizzazione asincrona verso il backend.

03. Scelta dell'Approccio

Per soddisfare i vincoli tecnologici e le direttive del corso, il sistema adotta un approccio basato su **Message Broker**, specificamente implementato tramite protocollo **MQTT** (Mosquitto).

3.1 Vantaggi del Message Broker (MQTT)

- **Pattern Publish/Subscribe:** Disaccoppia completamente i produttori di dati (sensori IoT/Simulatori) dai consumatori (motore di regole sul nodo Edge).
- **Leggerezza e Basso Overhead:** L'header minimale del protocollo MQTT garantisce basse latenze sulla rete locale (LAN) del pub.
- **Superamento dei NAT:** Il protocollo supporta connessioni persistenti (keep-alive). Una volta che il nodo Edge (client) si connette al broker MQTT nel Cloud, il canale rimane aperto in modo bidirezionale senza richiedere configurazioni di rete nel locale.
- **Quality of Service (QoS):** L'utilizzo di QoS 1 o 2 garantisce la consegna dei messaggi critici (es. fine partita, transazioni di punteggio) anche in caso di disconnessioni temporanee.

04. Architettura del Software

L'architettura del sistema *BitPub* è divisa in tre macro-componenti: Nodi IoT/Simulati, Edge Node e Cloud Microservices.

4.1 Componenti del Sistema

Livello	Modulo / Servizio	Responsabilità Principali
IoT / Sensors	BitPub-Simulators	Simulatori di eventi per Biliardo, Calciobalilla e Freccette. Generano payload MQTT raw pubblicati sui topic locali.
Edge	BitPub-Edge	Nodo locale Spring Boot. Include RuleEngineService per la logica real-time, BufferedEventRepository per lo stoccaggio locale offline e OfflineSyncScheduler per sincronizzare i match con il Cloud appena la

Livello	Modulo / Servizio	Responsabilità Principali
		connessione si ripristina.
Cloud Gateway	gateway-service, auth-service	Il Gateway Instrada le richieste. L'Auth emette e valida token JWT (JwtAuthenticationFilter).
Cloud Domain	match-service, tournament-service	Gestione centralizzata del ciclo di vita dei match, persistenza su DB cloud, organizzazione tabelloni dei tornei.
Cloud Data	statistics-service, game-catalog-service, locale-service	Gestione classifiche (Leaderboard), catalogo dei tipi di gioco supportati, anagrafica dei locali e delle postazioni fisiche (GameInstance).
Frontend	BitPub-WebApp (React/Vite)	Pannello di controllo Web per la gestione amministrativa e visualizzazione della <i>Live Match View</i> e della <i>Dashboard</i> utente.

05. Descrizione dell'Implementazione

L'implementazione è stata curata utilizzando l'ecosistema **Spring Boot** per i backend Java, standardizzando la comunicazione REST tra i microservizi e MQTT per i sensori. Di seguito si

riportano gli artefatti UML che descrivono l'implementazione del dominio e delle interazioni.

5.1 Class Diagram (Dominio Principale)

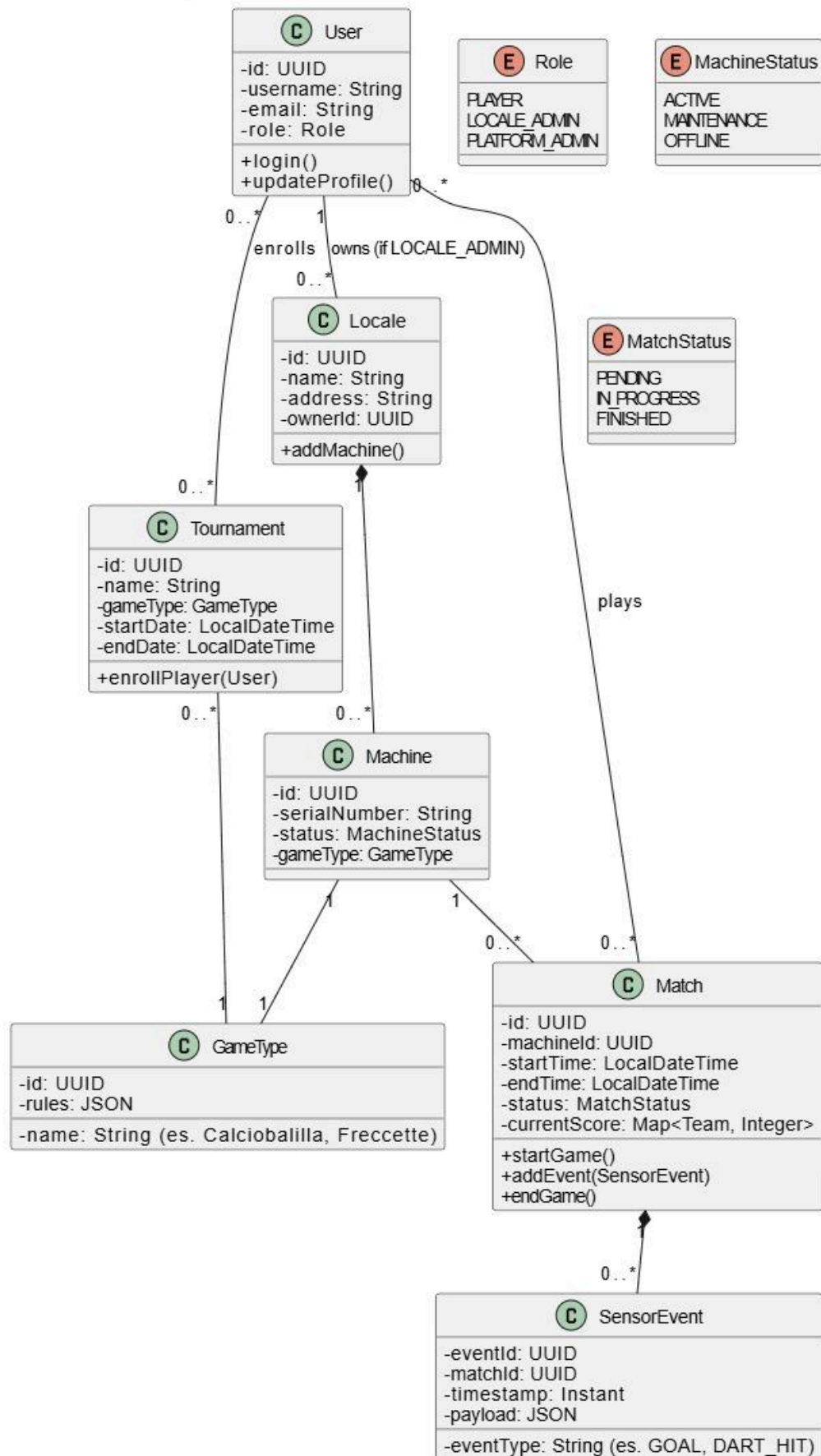
Il diagramma delle classi modella il nucleo del dominio di **BitPub**, definendo le entità fondamentali e le loro interrelazioni. Il modello è stato progettato per garantire l'integrità dei dati, supportando sia l'operatività locale (real-time) che la storicizzazione centralizzata nel Cloud.

Le classi principali identificate sono:

- **User**: Rappresenta l'identità dell'utente (Giocatore o Gestore). Gestisce l'autenticazione e i profili personali associati allo storico delle performance.
- **Locale**: Entità fisica che ospita le infrastrutture. È legata al concetto di "Luogo" ed è il contenitore gerarchico per le istanze di gioco.
- **GameInstance**: Rappresenta il tavolo o la macchina da gioco specifica (es. Biliardo, Calciobalilla). Contiene informazioni sullo stato operativo (ONLINE/OFFLINE) e riferimenti al nodo Edge associato.
- **Match**: Il cuore dell'attività di sistema. Questa classe registra gli eventi, i punteggi parziali, il timestamp di inizio/fine e l'esito della sfida. È collegata direttamente a **GameInstance** e **User**.
- **Tournament**: Aggregazione logica di più **Match**. Gestisce la struttura del tabellone e le classifiche temporanee.
- **Statistics**: Classe derivata che aggrega i dati dei **Match** completati. È progettata per una consultazione ottimizzata, permettendo al sistema di generare classifiche (Leaderboards) senza dover scansionare l'intera cronologia delle partite.

Il diagramma riflette un'architettura dove la persistenza è normalizzata per evitare ridondanze, assicurando che le sincronizzazioni asincrone tra il **BitPub-Edge** e il **BitPub-Cloud** mantengano la coerenza del modello dati anche in caso di disconnessioni prolungate.

BitPub Domain Model



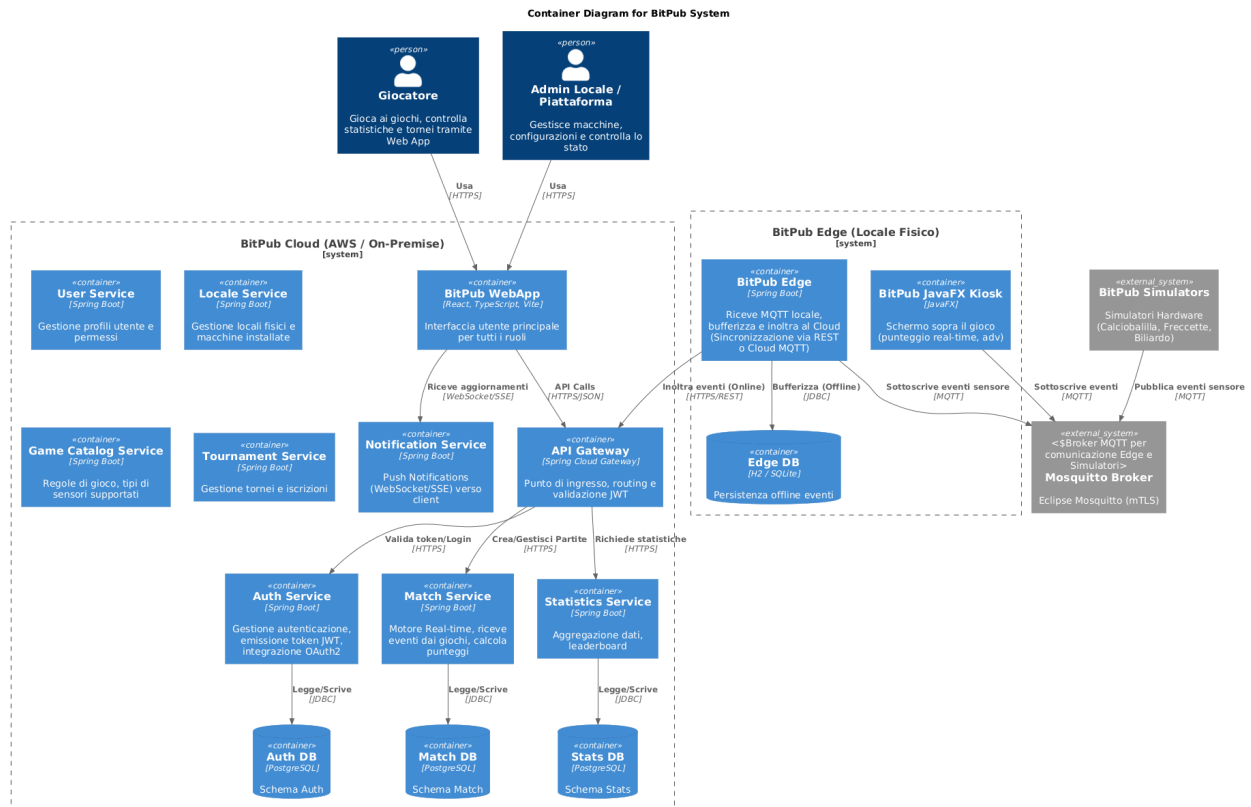
5.2 Diagramma dei package

Il diagramma dei package illustra la scomposizione modulare del sistema **BitPub**, evidenziando la netta separazione tra le componenti che risiedono nel nodo periferico (Edge) e quelle distribuite nel Cloud. La struttura è organizzata per garantire disaccoppiamento, manutenibilità e facilità di deployment.

La suddivisione segue tre macro-aree logiche:

- **Package** bitpub-edge: Contiene la logica operativa locale. Include i moduli per la gestione degli eventi (event-engine), la persistenza locale in assenza di connessione (buffer-repository) e il gestore delle sincronizzazioni verso il cloud (sync-scheduler). Questo package è progettato per essere autonomo e resiliente.
- **Package** bitpub-cloud: Raggruppa i microservizi centralizzati, organizzati secondo il pattern architetturale a domini.
 - api-gateway: Punto di ingresso unificato per il traffico pubblico.
 - auth-service: Gestione dell'identità e validazione dei token JWT.
 - match-service & tournament-service: Logica di business per il controllo dei match e la gestione dei tornei.
 - data-services: Moduli dedicati alla persistenza e alla consultazione di statistiche (statistics-service) e anagrafiche (locale-service, game-catalog-service).
- **Package** bitpub-webapp: Contiene il front-end client-side (React/Vite), organizzato in componenti riutilizzabili che consumano le API esposte dal Cloud.

Le dipendenze tra i package sono minimizzate: il bitpub-edge interagisce con il broker MQTT, mentre il bitpub-cloud comunica tramite REST, garantendo che i servizi possano evolvere indipendentemente l'uno dall'altro.



5.3 Sequence Diagram: Flusso Autenticazione (Auth Flow)

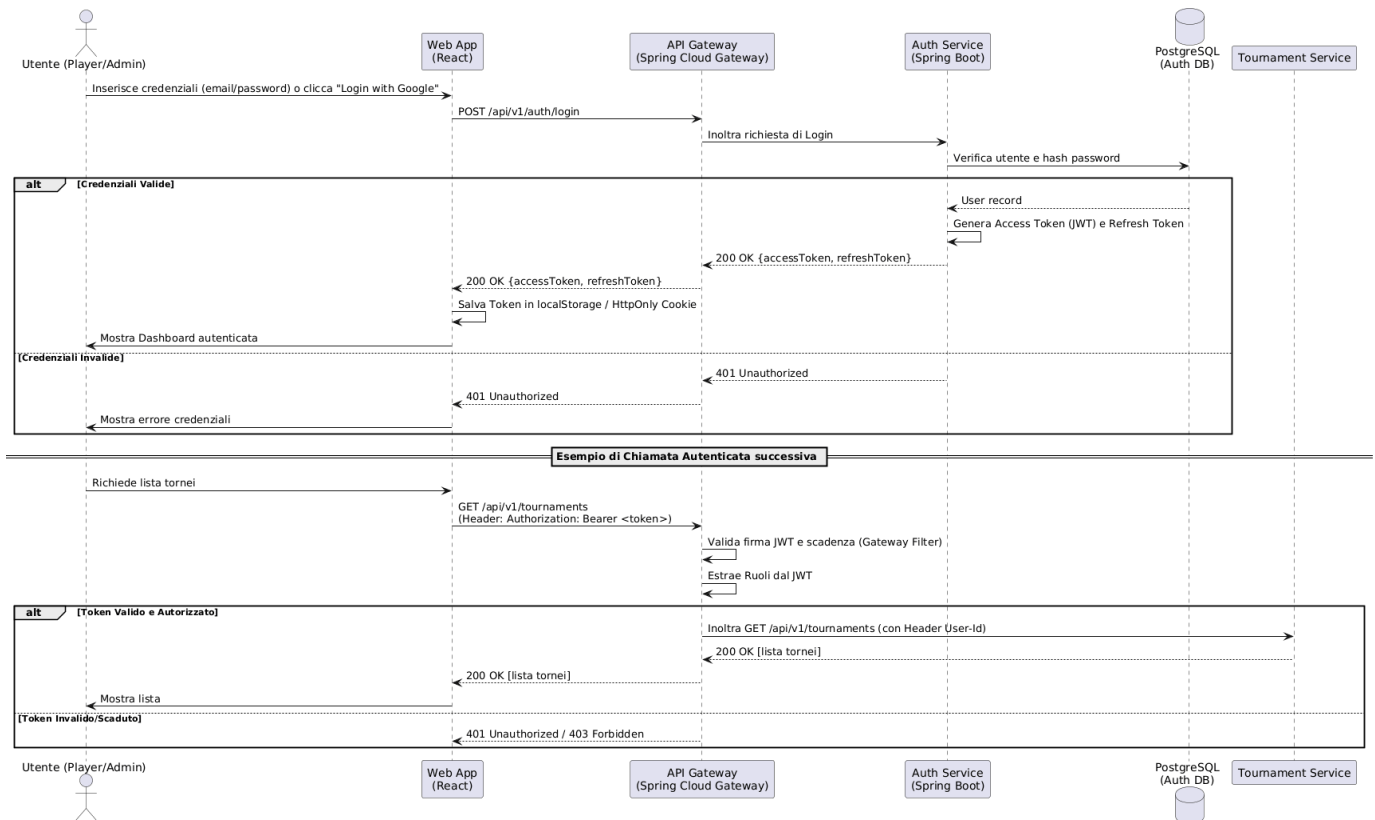
Il diagramma di sequenza illustra il meccanismo di autenticazione e autorizzazione del sistema **BitPub**, basato sul protocollo **JWT (JSON Web Token)**. Il flusso garantisce che ogni richiesta verso i microservizi protetti sia validata correttamente tramite l'intercettazione del `JwtAuthenticationFilter`.

Il processo segue questi passaggi logici:

- Richiesta di Login:** L'utente invia le proprie credenziali (username/password) tramite la WebApp all'endpoint di autenticazione esposto dall'Auth-Service.
- Validazione Credenziali:** L'Auth-Service interroga il database per verificare la corrispondenza delle credenziali.
- Generazione Token:** Una volta validato l'utente, l'Auth-Service genera un token JWT firmato, che viene restituito al client.
- Accesso alle Risorse:** Per ogni successiva chiamata alle API protette (es. Match-Service o Tournament-Service), la WebApp include il token nell'intestazione HTTP Authorization: Bearer <token>.
- Filtro e Verifica:** Il `JwtAuthenticationFilter`, configurato nell'API-Gateway, intercetta la richiesta, estrae il token e ne verifica la firma e la validità temporale.
- Risposta Autorizzata:** Se il token è valido, il Gateway inoltra la richiesta al

microservizio di destinazione; in caso contrario, viene restituito un errore 401 Unauthorized.

Questo approccio stateless garantisce la scalabilità del sistema, delegando la sicurezza a un punto centrale e sollevando i singoli microservizi dalla gestione diretta delle sessioni.



5.4 Sequence Diagram: Flusso Eventi di Gioco (Match Event Flow)

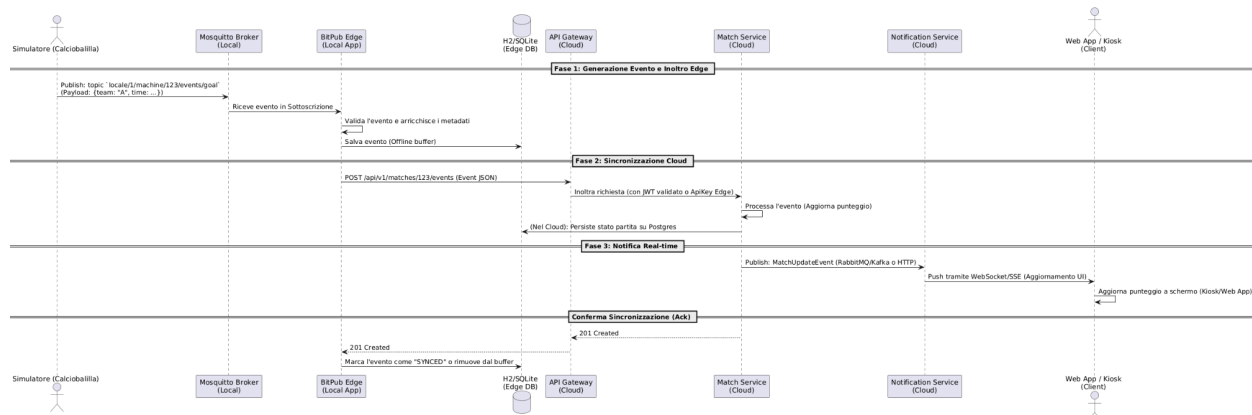
Il diagramma di sequenza illustra il flusso asincrono e la gestione degli eventi nel sistema **BitPub**, dal rilevamento fisico del sensore fino alla persistenza definitiva nel Cloud. L'architettura segue il paradigma Publish/Subscribe, progettato per garantire il real-time locale anche in assenza di connettività Internet.

Il processo si articola nelle seguenti fasi:

1. **Pubblicazione Evento:** Il simulatore hardware (o sensore IoT) pubblica un messaggio JSON sul topic MQTT del **Broker locale** al verificarsi di un evento (es. un punto segnato o fine partita).
2. **Elaborazione Locale (Real-time):** Il
3. **RuleEngineService**

4. , in ascolto sul topic, riceve l'evento, valida la mossa (secondo le regole del gioco) e aggiorna immediatamente lo stato del match locale per fornire feedback immediato al giocatore.
5. **Persistenza Offline:** L'evento validato viene salvato nel
6. BufferedEventRepository
7. . Questo passaggio è cruciale per garantire la resilienza: in caso di caduta della connessione internet, i dati rimangono persistiti localmente sul nodo Edge.
8. **Sincronizzazione Asincrona:** L'
9. OfflineSyncScheduler
10. monitora il buffer locale. Non appena viene rilevata connettività verso l'esterno, il modulo invia in modo sicuro i dati accumulati verso l'endpoint esposto dall'API Cloud.
11. **Consolidamento nel Cloud:** L'API Gateway riceve i dati, li valida tramite il
12. JwtAuthenticationFilter
13. e li inoltra al
14. Match-Service
15. , che scrive definitivamente le informazioni nel database centralizzato, rendendo il match visibile nelle statistiche globali e nelle classifiche.

Questo modello a disaccoppiamento garantisce che l'esperienza di gioco non sia mai interrotta da latenze di rete o disconnessioni, delegando la consistenza finale del sistema alla sincronizzazione asincrona.



5.5 Diagramma dei casi d'uso

Il diagramma dei casi d'uso illustra le interazioni principali tra gli attori del sistema (**Giocatore**, **Gestore del Locale**, **Sistema**) e le funzionalità offerte dalla piattaforma BitPub.

Gli attori principali identificati sono:

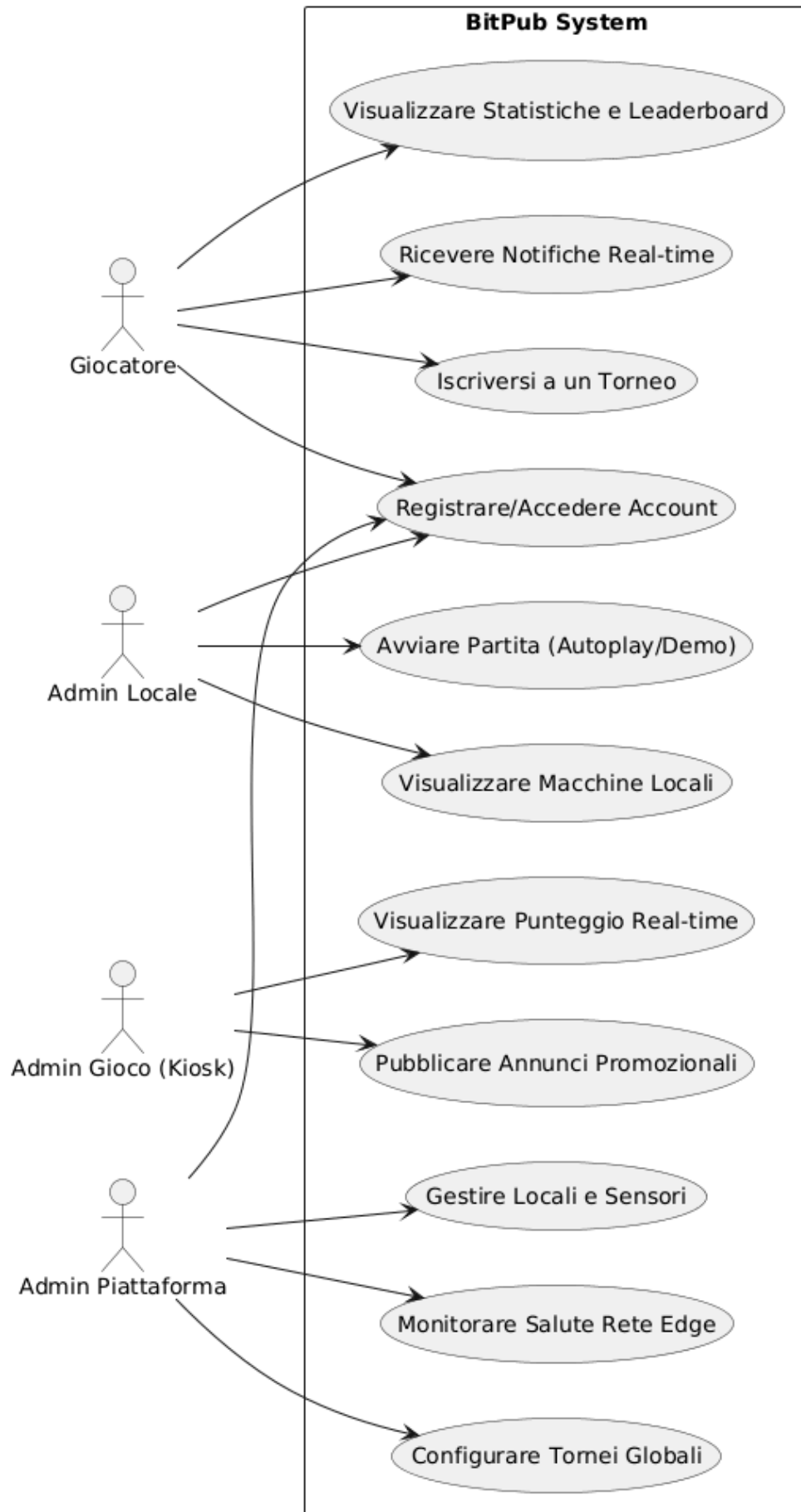
- **Giocatore:** L'utente che interagisce con il sistema per visualizzare le proprie statistiche,

monitorare i match in tempo reale e partecipare ai tornei.

- **Gestore del Locale:** L'utente amministratore che gestisce le istanze di gioco (Game Instances) fisicamente installate nel pub, configurando le postazioni e monitorando lo stato di connessione dei nodi Edge.

I casi d'uso principali includono:

- **Gestione Partita (Real-time):** Il sistema registra automaticamente gli eventi di gioco (punteggi, avanzamento partita) tramite i sensori IoT. Il Giocatore può visualizzare l'andamento in tempo reale tramite la **LiveMatchView**.
- **Accesso ai Servizi Cloud:**
 - *Visualizzazione Statistiche:* Il Giocatore consulta lo storico delle proprie prestazioni.
 - *Gestione Tornei:* Il Giocatore può registrarsi a tornei attivi e visualizzare le classifiche globali (Leaderboards).
- **Amministrazione Locale:** Il Gestore del Locale utilizza la WebApp per monitorare lo stato di salute dei nodi Edge ("ONLINE"/"OFFLINE") e configurare le impostazioni delle macchine da gioco.
- **Sincronizzazione Dati:** Processo automatizzato che avviene in background, dove il nodo Edge sincronizza i dati accumulati offline con il Cloud, garantendo la coerenza informativa anche in caso di disconnessione temporanea.



5.6 Interfaccia Utente (UI)

L'interfaccia utente, situata nei pacchetti BitPub-WebApp e demo-control-panel, è sviluppata in **React** con build tool **Vite** e styling basato su **Tailwind CSS**. Le viste principali includono:

- **DashboardPage:** Panoramica dello stato dell'utente, ultimi tornei giocati e accesso rapido alle statistiche.
- **LiveMatchView:** Componente reattivo (sfrutta l'integrazione WebSockets/SSE) che mostra in tempo reale i punteggi del tavolo mentre i sensori Edge registrano gli eventi.
- **LeaderboardPage:** Interfaccia tabellare che interroga lo statistics-service per mostrare i ranking globali e locali.
- **LocalesPage:** Vista dedicata ai gestori per visualizzare lo stato operativo ("ONLINE", "OFFLINE") dei nodi BitPub-Edge installati nei rispettivi pub.

06. Validazione

Per assicurare la qualità e la stabilità del software, è stata definita una robusta pipeline di test che copre tutti i livelli della Piramide dei Test.

- **Test Unitari:** Ciascun microservizio (es. user-service, match-service) include classi di test (es. UserServiceTest, RuleEngineServiceTest) per validare la logica di business isolata mediante framework *JUnit 5* e *Mockito*.
- **Test di Integrazione:** Sono state sviluppate suite di test come EdgeMqttIntegrationTest e UserRepositoryIntegrationTest utilizzando *Testcontainers*. Questo garantisce che i microservizi dialoghino correttamente con i database (PostgreSQL/MySQL) e il broker MQTT reali caricati in container effimeri durante la build.
- **Test End-to-End (E2E):** Procedure come UserServiceE2ETest simulano l'intero ciclo di vita dell'utente passando attraverso i layer REST del framework Spring.
- **Collaudo di Sistema e Deployment:** Il collaudo dell'intero ecosistema è orchestrato da docker-compose.yml alla radice del progetto. Gli script PowerShell demo.ps1 e rebuild_no_cache.ps1 permettono l'avvio immediato della simulazione locale, dimostrando il corretto interfacciamento tra i simulatori (biliardo, freccette), il nodo Edge e la suite Cloud.

Documento redatto in conformità alle specifiche di progetto PISSIR per la valutazione finale.